CT2003 Restaurant Rush

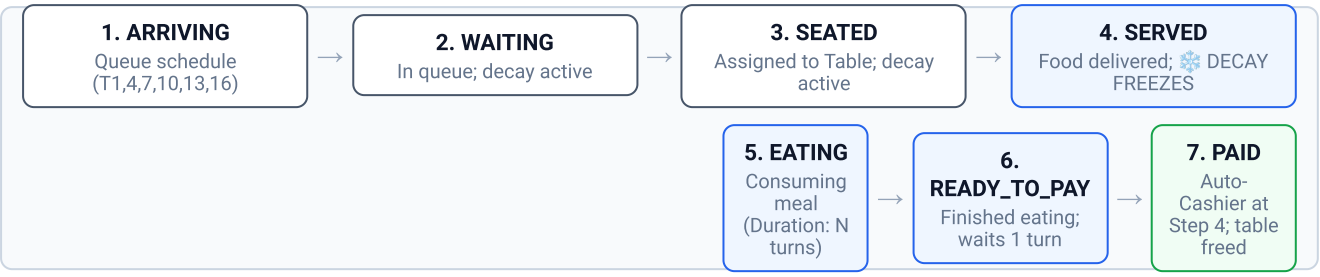
System Architecture, Decoupled Domain & Customer Lifecycle Specification

STAGE 1–3 COMPLETE

Document Changelog & Team Review Additions

- **Customer Lifecycle Expansion:** Formally integrated all 7 Customer States (`ARRIVING` through `PAID`) into the core architecture diagram and domain rules.
- **Satisfaction Decay Freeze Rule:** Added visual indicator and explicit specification showing decay instantly halting upon entering `SERVED` .
- **Turn 16 Customer 6 Boundary Constraint:** Documented the mathematical proof for Customer 6 being unserviceable prior to Turn 18 end.
- **Exception Handling & State Transitions:** Defined the contract for `InvalidGameActionException` to prevent illegal state transitions without consuming player turns.

1 7-STATE CUSTOMER PROGRESSION LIFECYCLE



2 COMPLETE CUSTOMER STATE SPECIFICATION

STATE ENUM	TRANSITION TRIGGER	DECAY STATUS	ARCHITECTURAL BEHAVIOR & CAPACITY RULES
`ARRIVING`	Turn Start Match	Inactive (100)	Stored in arrival schedule buffer. Does not occupy queue capacity.
`WAITING`	Turn matching arrival	Active (-8/turn)	Enters waiting queue. Host or Waiter can transition to SEATED.
`SEATED`	Waiter/Host assign	Active (-8/turn)	Binds to exact 1 Table (Capacity: 1). Table is occupied.
`SERVED`	Waiter delivers food	**FROZEN (0)**	**Satisfaction freezes instantly.** Order status becomes SERVED.
`EATING`	Automatic next turn	**FROZEN (0)**	Customer consumes food over meal duration (1 turn in Stage 1).
`READY_TO_PAY`	Meal completion	**FROZEN (0)**	Waits for automated Cashier step. Table remains locked.
`PAID`	Cashier Step 4 auto	Terminal (Saved)	Revenue added to ledger; Customer removed; Table cleared.

3 DECOUPLED 2-BRANCH MEDIATOR ARCHITECTURE

The system enforces zero direct coupling between the **Staff Branch** and the **Customer Branch**. All interactions are filtered through the `GameController` Mediator. Entities communicate solely by updating domain state containers (`Table` , `Order`), preventing illegal dependency cross-overs.

4 KEY ENTITY & SERVICE CLASS SPECIFICATIONS

Customer (Domain Entity)

Attributes: `id` , `satisfaction` (int), `state` (CustomerState), `decayRate` (int).
Key Behavior: `decaySatisfaction()` reduces satisfaction only when state is `WAITING` or `SEATED` . Throws exception on invalid state transition.

GameController (Mediator / Logic)

Attributes: `currentTurn` (1-18), `totalRevenue` , `tables` , `queue` .
Key Behavior: Coordinates turn execution cycle, handles player choices for Waiter/Chef, auto-executes Cashier at Step 4, enforces arrival schedules.

Table & Order Entities

Attributes: `tableNo` , `occupiedBy` , `prepUnits` , `requiredUnits` .
Key Behavior: Strict capacity (1 Table = 1 Customer). Holds order state transitions (`PENDING` → `PREPARING` → `READY` → `SERVED`).

Staff Entities (Waiter, Chef, Cashier)

Attributes: `isBusy` , `capacityPerTurn` .
Key Behavior: Waiter handles 1 task/turn. Chef adds 1 prep unit/turn. Cashier automatically sweeps `READY_TO_PAY` tables during Step 4.

5 EXCEPTION HANDLING PROTOCOL

EXCEPTION CLASS	TRIGGER CONDITION	SYSTEM ACTION & TURN RECOVERY
`InvalidGameActionException`	Player commands action on invalid state (e.g., serving uncooked order).	Action rejected. Turn capacity is **NOT consumed**. Player re-promoted.
`TableOccupiedException`	Attempting to seat a customer at a table with `isOccupied == true` .	Assignment blocked. Requires selecting an empty table or waiting.
`InvalidStateTransitionException`	Direct state transition skip (e.g., `WAITING` directly to `SERVED`).	Domain entity halts mutation. Error logged to GameController boundary layer.

6 18-TURN SIMULATION LIFECYCLE MATRIX

TURN	ARRIVALS	WAITER ACTION (PLAYER)	CHEF ACTION (PLAYER)	CASHIER ACTION (AUTO)	CUSTOMER STATE PROGRESSION
T1	Cust 1 arrives	Seat Cust 1 → T1	Take / Start Order 1	—	Cust 1: `ARRIVING` → `WAITING` → `SEATED`
T2	—	Take Order / Wait	Cook Order 1 (1/1)	—	Order 1 status becomes `READY`
T3	—	Serve Cust 1	Idle / Prep next	—	Cust 1: `SERVED` ❄ (Decay halts)
T4	Cust 2 arrives	Seat Cust 2 → T2	Cook Order 2	—	Cust 1: `EATING` ; Cust 2: `WAITING` → `SEATED`
T5	—	Serve Cust 2	Idle	Cust 1 auto-paid	Cust 1: `READY_TO_PAY` → `PAID` (T1 freed)
T7..15	Cust 3,4,5	Cycle seating/serving	Cycle prep	Step 4 sweep	Cust 2–5 transition through eating to `PAID`
T16..18	Cust 6 (T16)	Seat / Take Order 6	Cook Order 6	Final Sweeps	Cust 6 reaches `SERVED` / `EATING` by T18 end.

7 CUSTOMER 6 (TURN 16 ARRIVAL) BOUNDARY PROOF

Mathematical Proof of Unserviceable Constraint

Customer 6 enters the queue at **Turn 16**. The absolute fastest execution sequence requires:
• **Turn 16:** Waiter assigns Cust 6 to table (`SEATED`). Chef takes order.
• **Turn 17:** Chef prepares dish (`READY`). Waiter delivers food (`SERVED`).
• **Turn 18:** Cust 6 eats meal (`EATING`) and transitions to `READY_TO_PAY` at turn conclusion.
Result: Cashier auto-payment runs at Step 4 of each turn. Because Cust 6 only reaches `READY_TO_PAY` at the end of Turn 18, payment can only process on **Turn 19** (which does not exist). Thus, Cust 6 mathematically cannot reach `PAID` state before the 18-turn game ends.

8 DESIGN RECOMMENDATIONS FOR TEAM INTEGRATION

When implementing the `Customer` entity, enforce state checks inside `decaySatisfaction()` using an explicit `EnumSet` of decaying states (`WAITING` , `SEATED`). Ensure unit tests verify that calling decay on `SERVED` or `EATING` customers yields a net `0` change in satisfaction value.